Рефакторинг модуля Auth на CQRS

Пошаговый конспект: что установить, что добавить, что изменить и что удалить

Репозиторий: `ShipaM/video-meetings` (ветка `master`) · приложение `apps/api` · NestJS 12 · Prisma 7 · Vitest

Составлено 13.10.2026 по коду из репозитория

> **Источник.** Весь конспект построен на файлах, которые я прочитал в `master`: `auth.module.ts`, `auth.controller.ts`, `auth.service.ts`, `dto/auth-credentials.dto.ts`, `app.module.ts`, `main.ts`, `prisma.service.ts`, `schema.prisma`, `auth.e2e-spec.ts`, оба `package.json`, `tsconfig.json`, vitest-конфиги и оба `CLAUDE.md`. Если локально у тебя есть незакоммиченные правки, сверяй код с ними. Дерево папок `apps/api/src` через GitHub я целиком не видел, поэтому имена файлов взяты из импортов.

Что в итоге получится

- `POST /auth/register` и `POST /auth/login` работают **точно так же**: те же тела запросов, те же коды 201 / 400 / 401 / 409, те же JWT.
- Контроллер больше не знает про бизнес-логику. Он собирает команду или запрос и отдаёт в шину (`CommandBus` / `QueryBus`).
- Регистрация (меняет состояние) = **команда** + **command-handler**. Логин (только читает и выдаёт токен) = **query** + **query-handler**.
- `AuthService` удаляется. Выдача токена выносится в маленький `TokenService`, чтобы её использовали оба обработчика.
- Твой `test/auth.e2e-spec.ts` **не меняется**. Он и есть страховочная сетка: если после каждого шага он зелёный, рефакторинг безопасен.

Как это работает (схема)

```
HTTP POST /auth/register
  -> AuthController.register(dto)
     -> commandBus.execute(new RegisterUserCommand(email, password))
        -> RegisterUserHandler.execute()   [Prisma: create user, TokenService: issue JWT]

HTTP POST /auth/login
  -> AuthController.login(dto)
     -> queryBus.execute(new LoginQuery(email, password))
        -> LoginHandler.execute()          [Prisma: find user, bcrypt compare, JWT]
```

Зачем переходить на CQRS: почему так лучше

Сейчас в `AuthService` две операции, и он выглядит нормально. Проблемы CQRS решает не сегодня, а тогда, когда в проекте станет много фич. У `video-meetings` они почти наверняка будут: комнаты, участники, приглашения, запись, чат. Ниже причины, по которым паттерн окупается, а потом честно про цену.

Что получаешь

Причина	Что это значит на практике в твоём проекте
1. Один handler = одна задача	Сейчас `AuthService` собирает всё подряд. Добавишь сброс пароля, refresh-токен, выход, смену e-mail, и он вырастет в «божественный класс» на 300+ строк с кучей зависимостей. С CQRS каждая операция это отдельный маленький файл со своими зависимостями.
2. Тонкий контроллер	Контроллер только превращает DTO в сообщение и отправляет в шину. Нет бизнес-логики в HTTP-слое, поэтому его не нужно менять при смене логики.
3. Логика не привязана к HTTP	Видеовстречи это realtime. Когда появится WebSocket-шлюз (сигналинг, вход в комнату), он вызовет тот же `commandBus.execute(new JoinRoomCommand(...))`, что и REST-контроллер. Один handler обслужит HTTP, WebSocket, cron-задачу и CLI-скрипт без копипаста.
4. Простое тестирование	Handler создаётся через `new` с двумя моками (шаг 10 гайда). Не нужен `Test.createTestingModule`, HTTP и база. Тесты быстрые и читаются как спецификация.
5. Код читается как список возможностей	`RegisterUserCommand`, `LoginQuery`, `JoinRoomCommand`, `GetMeetingQuery`: названия на языке предметной области. Открыл папку `commands/` и сразу видно, что умеет приложение.
6. Меньше конфликтов в git	Разные фичи лежат в разных файлах. Два разработчика не правят один раздутый сервис одновременно.
7. Готовая основа для роста	Шина даёт точки расширения без переписывания: события (`UserRegisteredEvent` -> письмо, аналитика), саги («встреча создана» -> «создать комнату»), сквозная логика (логирование, транзакции, метрики) на уровне шины.
8. Чтение и запись можно развивать отдельно	Запросы можно позже ускорить кэшем или read-репликой БД, не трогая команды. Это главный «взрослый» плюс CQRS, но он нужен не сразу.

Было и стало (для auth)

	Сейчас (сервис)	После (CQRS)
Где логика регистрации	метод `AuthService.register`	`RegisterUserHandler.execute`
Где логика логина	метод `AuthService.login`	`LoginHandler.execute`
Что знает контроллер	конкретный `AuthService`	только `CommandBus` и `QueryBus`
Добавить «сброс пароля»	ещё метод в общий сервис	новый `ResetPasswordCommand` + handler, старые файлы не трогаем
Тест логики	`Test.createTestingModule` или e2e	`new Handler(mock, mock)` без фреймворка
Видно ли возможности системы	нужно читать сервис целиком	видно по именам файлов в `commands/` и `queries/`

Цена (честно)

Важно понимать. CQRS в `@nestjs/cqrs` это **лёгкая версия** паттерна: одна база, одна модель данных, просто разделение на «записывающие» и «читающие» классы. Отдельные БД для чтения и записи, event sourcing и eventual consistency тебе **не нужны** и в этом рефакторинге не появляются.

- **Больше файлов.** Один `AuthService` превращается в ~8 небольших файлов (команда, query, два handler, `TokenService`, индексы). Это и есть плата за разделение.
- **Больше «церемоний».** Чтобы вызвать логику, нужно создать объект сообщения и отправить в шину, а не просто вызвать метод. Навигация «перейти к определению» из контроллера ведёт в шину, а не в handler (ищется по имени команды).
- **Для двух эндпоинтов это перебор.** Если бы приложение оставалось таким маленьким, обычный сервис был бы проще. CQRS выигрывает по мере роста, а не на старте.
- **Граница «команда или запрос» иногда спорная.** Логин меняет только токен, а не данные, поэтому он query, но многие команды делают его командой. Выбери одно правило и держись его во всём проекте.

Рекомендация. Для `video-meetings` вывод такой: приложение будет расти (realtime, комнаты, участники), поэтому переход **оправдан и дешевле сделать сейчас, пока в auth всего два use case**. Auth при этом отличная песочница, чтобы отработать структуру `commands/ queries/ handlers/`, а новые модули сразу строить по ней.

0. Что у тебя сейчас (отправная точка)

Файл (`apps/api/src/...`)	**Что делает сейчас**
`auth/auth.controller.ts`	`@Controller("auth")`, два метода вызывают `AuthService.register` / `login`
`auth/auth.service.ts`	Вся логика: bcrypt (10 раундов), Prisma, ловит `P2002` -> 409, `issueToken` через `JwtService`
`auth/auth.module.ts`	`JwtModule.registerAsync` (секрет из `JWT_SECRET`), `controllers: [AuthController]`, `providers: [AuthService]`
`auth/dto/auth-credentials.dto.ts`	`RegisterDto` и `LoginDto` (class-validator). **Остаются без изменений**
`app.module.ts`	`imports: [PrismaModule, AuthModule]`
`main.ts`	Глобальный `ValidationPipe({ whitelist: true })`. **Не трогаем**
`test/auth.e2e-spec.ts`	11 e2e-тестов на реальной БД (supertest). **Не трогаем**

> **Внимание.** В репозитории `"type": "module"` и `module: nodenext`, поэтому **в каждом относительном импорте обязательно расширение** `.js` (например `./token.service.js`), даже если файл на самом деле `.ts`. Все примеры ниже уже так написаны. Комментарии в коде и сообщения коммитов пиши по-английски: так требует `CLAUDE.md` репозитория.

Целевая структура папки `apps/api/src/auth`

```
auth/
├── auth.constants.ts                  NEW   BCRYPT_ROUNDS
├── auth.types.ts                      NEW   AccessTokenResult
├── token.service.ts                   NEW   issue(userId, email) -> { accessToken }
├── auth.controller.ts                 EDIT  uses CommandBus / QueryBus
├── auth.module.ts                     EDIT  providers: TokenService + handlers
├── auth.service.ts                    DELETE (last step)
├── dto/
│   └── auth-credentials.dto.ts        unchanged
├── commands/
│   ├── register-user.command.ts       NEW
│   ├── index.ts                       NEW   CommandHandlers = [...]
│   └── handlers/
│       ├── register-user.handler.ts   NEW
│       └── register-user.handler.spec.ts   NEW (optional unit test)
└── queries/
    ├── login.query.ts                 NEW
    ├── index.ts                       NEW   QueryHandlers = [...]
    └── handlers/
        ├── login.handler.ts           NEW
        └── login.handler.spec.ts      NEW (optional unit test)
```

Шаг 0. Подготовка и страховочная сетка

1. Убедись, что версия Node подходит: `node -v`. Пакет `@nestjs/cqrs` 12.1.0 требует Node `^20.19.0` или `>=22.12.0` (в корневом `package.json` написано лишь `>=20`, так что проверь минорную версию).
2. Подними базу и накати миграции, чтобы e2e-тесты могли работать.
3. Создай ветку для рефакторинга.
4. Прогони e2e-тесты **до** любых изменений. Они должны быть зелёными.

```
# from the repository root
node -v
npm run db:up
npm run prisma:migrate --workspace=api
git checkout -b refactor/auth-cqrs
npm run test:e2e --workspace=api
```

Заметка. Если `.env` ещё не создан, скопируй `apps/api/.env.example` в `apps/api/.env` и проверь `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRES_IN`.

Проверка. Все 11 тестов в `auth.e2e-spec.ts` проходят. Если нет, остановись и почини окружение, иначе потом не поймёшь, что сломал рефакторинг.

Шаг 1. Установить зависимость

Нужен один пакет. `rxjs` и `reflect-metadata` у тебя уже есть, это peer-зависимости CQRS-модуля.

```
# from the repository root
npm install @nestjs/cqrs --workspace=api
```

- В `apps/api/package.json` в `dependencies` появится `"@nestjs/cqrs": "^12.1.0"`, а `package-lock.json` в корне обновится.
- Совместимость: на 13.10.2026 последняя версия `@nestjs/cqrs` 12.1.0, peer-зависимости `@nestjs/core` и `@nestjs/common` `^12.0.0`, пакет ESM-совместимый (`"type": "module"`).
- Больше **ничего** ставить не нужно: ни `@nestjs/event-emitter`, ни `swc`.

Проверка. `npm run build --workspace=api` проходит без ошибок.

Шаг 2. Создать папки

```
# macOS / Linux / Git Bash
mkdir -p apps/api/src/auth/commands/handlers apps/api/src/auth/queries/handlers

# Windows PowerShell
New-Item -ItemType Directory -Force apps/api/src/auth/commands/handlers, apps/api/src/auth/queries/handlers
```

Шаг 3. Общие кусочки: константа, тип результата, TokenService

Сейчас `issueToken` и `BCRYPT_ROUNDS` живут внутри `auth.service.ts`. Мы выносим их, чтобы handlers были тонкими, а код не дублировался.

apps/api/src/auth/auth.constants.ts (NEW)

```
export const BCRYPT_ROUNDS = 10;
```

apps/api/src/auth/auth.types.ts (NEW)

```
export interface AccessTokenResult {
  accessToken: string;
}
```

apps/api/src/auth/token.service.ts (NEW)

```
import { Injectable } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import type { AccessTokenResult } from './auth.types.js';

@Injectable()
export class TokenService {
  constructor(private readonly jwt: JwtService) {}

  async issue(userId: string, email: string): Promise<AccessTokenResult> {
    return { accessToken: await this.jwt.signAsync({ sub: userId, email }) };
  }
}
```

Проверка. `npm run build --workspace=api`. Пока это просто новые файлы, на работу приложения они не влияют.

Шаг 4. Команда RegisterUser и её command-handler

Команда это простой объект с данными и намерением («зарегистрируй пользователя»). Наследование от `Command<T>` типизирует результат, который вернёт `commandBus.execute()`.

apps/api/src/auth/commands/register-user.command.ts (NEW)

```
import { Command } from '@nestjs/cqrs';
import type { AccessTokenResult } from '../auth.types.js';

export class RegisterUserCommand extends Command<AccessTokenResult> {
  constructor(
    public readonly email: string,
    public readonly password: string,
  ) {
    super();
  }
}
```

Command-handler содержит логику, которую раньше выполнял `AuthService.register`. Код перенесён один в один: нормализация e-mail, bcrypt, `P2002` -> 409.

apps/api/src/auth/commands/handlers/register-user.handler.ts (NEW)

```
import { ConflictException } from '@nestjs/common';
import { CommandHandler, ICommandHandler } from '@nestjs/cqrs';
import { hash } from 'bcryptjs';
import { Prisma } from '../../../generated/prisma/client.js';
import { PrismaService } from '../../../prisma/prisma.service.js';
import { BCRYPT_ROUNDS } from '../../auth.constants.js';
import type { AccessTokenResult } from '../../auth.types.js';
import { TokenService } from '../../token.service.js';
import { RegisterUserCommand } from '../register-user.command.js';

@CommandHandler(RegisterUserCommand)
export class RegisterUserHandler implements ICommandHandler<RegisterUserCommand> {
  constructor(
    private readonly prisma: PrismaService,
    private readonly tokens: TokenService,
  ) {}

  async execute({ email, password }: RegisterUserCommand): Promise<AccessTokenResult> {
    const normalized = email.toLowerCase();
    try {
      const user = await this.prisma.user.create({
        data: { email: normalized, passwordHash: await hash(password, BCRYPT_ROUNDS) },
      });
      return await this.tokens.issue(user.id, user.email);
    } catch (error) {
      if (error instanceof Prisma.PrismaClientKnownRequestError && error.code === 'P2002') {
        throw new ConflictException('User with this e-mail already exists');
      }
      throw error;
    }
  }
}
```

Индекс-файл собирает всё в массив, чтобы потом одной строкой подключить в модуль:

apps/api/src/auth/commands/index.ts (NEW)

```
import { RegisterUserHandler } from './handlers/register-user.handler.js';

export { RegisterUserCommand } from './register-user.command.js';

export const CommandHandlers = [RegisterUserHandler];
```

Заметка. Пути импортов в handler считаются от `src/auth/commands/handlers/`: `../../../generated` это `src/generated`, `../../auth.constants.js` это `src/auth/auth.constants.ts`. Если TypeScript ругается на путь, ты, скорее всего, положил файл не в ту папку.

Шаг 5. Query Login и её query-handler

Логин ничего не меняет в базе: он ищет пользователя, сверяет пароль и выдаёт токен. Поэтому по правилам CQRS это **запрос** (query), а не команда.

apps/api/src/auth/queries/login.query.ts (NEW)

```
import { Query } from '@nestjs/cqrs';
import type { AccessTokenResult } from '../auth.types.js';

export class LoginQuery extends Query<AccessTokenResult> {
  constructor(
    public readonly email: string,
    public readonly password: string,
  ) {
    super();
  }
}
```

apps/api/src/auth/queries/handlers/login.handler.ts (NEW)

```
import { UnauthorizedException } from '@nestjs/common';
import { IQueryHandler, QueryHandler } from '@nestjs/cqrs';
import { compare } from 'bcryptjs';
import { PrismaService } from '../../../prisma/prisma.service.js';
import type { AccessTokenResult } from '../../auth.types.js';
import { TokenService } from '../../token.service.js';
import { LoginQuery } from '../login.query.js';

@QueryHandler(LoginQuery)
export class LoginHandler implements IQueryHandler<LoginQuery> {
  constructor(
    private readonly prisma: PrismaService,
    private readonly tokens: TokenService,
  ) {}

  async execute({ email, password }: LoginQuery): Promise<AccessTokenResult> {
    const user = await this.prisma.user.findUnique({ where: { email: email.toLowerCase() } });
    if (!user || !(await compare(password, user.passwordHash))) {
      throw new UnauthorizedException('Invalid e-mail or password');
    }
    return this.tokens.issue(user.id, user.email);
  }
}
```

apps/api/src/auth/queries/index.ts (NEW)

```
import { LoginHandler } from './handlers/login.handler.js';

export { LoginQuery } from './login.query.js';

export const QueryHandlers = [LoginHandler];
```

Выбор дизайна. Если хочешь, чтобы логин был командой (`LoginUserCommand` + `CommandBus`), это тоже допустимо: многие команды считают выдачу токена побочным эффектом. Тогда положи файлы в `commands/`, а не в `queries/`. Для обучения я оставил query, чтобы ты прошёл обе ветки CQRS.

Шаг 6. Проверить, что всё компилируется

```
npm run build --workspace=api
npm run lint --workspace=api
```

Проверка. Сборка и линт без ошибок. Приложение всё ещё работает через старый `AuthService`: новые файлы пока никуда не подключены.

Шаг 7. Подключить CQRS: AppModule, AuthModule, AuthController

Внимание. Эти три файла правь **подряд, не запуская приложение между ними**. Если убрать `AuthService` из модуля, а контроллер ещё его просит, Nest не поднимется.

7.1. `app.module.ts`: добавь `CqrsModule.forRoot()` в `imports`.

apps/api/src/app.module.ts (EDIT)

```
import { Module } from '@nestjs/common';
import { CqrsModule } from '@nestjs/cqrs';
import { AppController } from './app.controller.js';
import { AppService } from './app.service.js';
import { AuthModule } from './auth/auth.module.js';
import { PrismaModule } from './prisma/prisma.module.js';

@Module({
  imports: [CqrsModule.forRoot(), PrismaModule, AuthModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

7.2. `auth/auth.module.ts`: заменить `AuthService` на `TokenService` и массивы handlers.

apps/api/src/auth/auth.module.ts (EDIT)

```
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { AuthController } from './auth.controller.js';
import { CommandHandlers } from './commands/index.js';
import { QueryHandlers } from './queries/index.js';
import { TokenService } from './token.service.js';

@Module({
  imports: [
    JwtModule.registerAsync({
      useFactory: () => ({
        secret: process.env.JWT_SECRET,
        signOptions: { expiresIn: (process.env.JWT_EXPIRES_IN ?? '7d') as '7d' },
      }),
    }),
  ],
  controllers: [AuthController],
  providers: [TokenService, ...CommandHandlers, ...QueryHandlers],
})
export class AuthModule {}
```

7.3. `auth/auth.controller.ts`: контроллер становится тонким диспетчером.

apps/api/src/auth/auth.controller.ts (EDIT)

```
import { Body, Controller, Post } from '@nestjs/common';
import { CommandBus, QueryBus } from '@nestjs/cqrs';
import type { AccessTokenResult } from './auth.types.js';
import { RegisterUserCommand } from './commands/index.js';
import { LoginDto, RegisterDto } from './dto/auth-credentials.dto.js';
import { LoginQuery } from './queries/index.js';

@Controller('auth')
export class AuthController {
  constructor(
    private readonly commandBus: CommandBus,
    private readonly queryBus: QueryBus,
  ) {}

  @Post('register')
  register(@Body() dto: RegisterDto): Promise<AccessTokenResult> {
    return this.commandBus.execute(new RegisterUserCommand(dto.email, dto.password));
  }

  @Post('login')
  login(@Body() dto: LoginDto): Promise<AccessTokenResult> {
    return this.queryBus.execute(new LoginQuery(dto.email, dto.password));
  }
}
```

Внимание. Код ответа логина по-прежнему **201** (по умолчанию для `@Post`), и твои e2e-тесты ждут именно его. Не добавляй `@HttpCode(200)`, иначе тест сломается.

Шаг 8. Удалить AuthService

1. Убедись, что на него больше никто не ссылается: `grep -rn "auth.service" apps/api/src apps/api/test` (на Windows: `Select-String -Path apps/api/src/**/*.ts -Pattern "auth.service"`).
2. Удали файл `apps/api/src/auth/auth.service.ts`.
3. Если был `auth.service.spec.ts`, удали и его (в `CLAUDE.md` он не упоминается, так что скорее всего его нет).

```
git rm apps/api/src/auth/auth.service.ts
```

Шаг 9. Полная проверка

```
npm run build --workspace=api
npm run lint --workspace=api
npm run test:e2e --workspace=api
npm run dev:api        # and try both routes manually if you want
```

Проверка. Те же 11 e2e-тестов зелёные без единой правки в тестах. Это и есть доказательство, что поведение API не изменилось.

Симптом	Причина и решение
`Nest can't resolve dependencies of the RegisterUserHandler (?, TokenService)`	Не хватает провайдера: проверь, что `TokenService` есть в `providers` модуля `AuthModule`, а `PrismaModule` подключён (он у тебя глобальный). Если ошибка только под Vitest, добавь явный токен в конструктор handler: `@Inject(PrismaService)`. Decorator metadata под Vitest иногда не эмитится.
`Nest can't resolve dependencies of the AuthController (?, QueryBus)`	Нет `CqrsModule.forRoot()` в `AppModule`. Если оставил так, импортируй `CqrsModule` прямо в `AuthModule`.

Симптом	Причина и решение
`CommandHandlerNotFoundException` / `QueryHandlerNotFoundException`	Handler не в `providers` или забыт декоратор `@CommandHandler(...)` / `@QueryHandler(...)`. Проверь `commands/index.ts` и `queries/index.ts`.
`ERR_MODULE_NOT_FOUND` при запуске	Забыл `.js` в относительном импорте.
TypeScript: у `Command<...>` / `Query<...>` нет такого экспорта	Версия `@nestjs/cqrs` ниже 11. Обнови до 12.x. Запасной вариант: обычный класс без `extends` и `commandBus.execute<Cmd, Result>(...)`.
ESLint `no-unsafe-*` в spec-файлах	Мокам нужен каст `as unknown as PrismaService` (см. шаг 10).
Предупреждение `EBADENGINE`	Обнови Node до `20.19+` или `22.12+`.

Шаг 10. Юнит-тесты handlers (по желанию, но стоит)

Главная выгода CQRS: каждый handler это маленький класс, который можно тестировать без HTTP и без базы. Создаёшь его напрямую через `new`, а зависимости подменяешь моками. Тест подхватится конфигом `vitest.config.ts` (`**/*.spec.ts`), запуск: `npm run test --workspace=api`.

apps/api/src/auth/commands/handlers/register-user.handler.spec.ts (NEW)

```
import { ConflictException } from '@nestjs/common';
import { Prisma } from '../../../generated/prisma/client.js';
import type { PrismaService } from '../../../prisma/prisma.service.js';
import type { TokenService } from '../../token.service.js';
import { RegisterUserCommand } from '../register-user.command.js';
import { RegisterUserHandler } from './register-user.handler.js';

describe('RegisterUserHandler', () => {
  const create = vi.fn();
  const issue = vi.fn().mockResolvedValue({ accessToken: 'token' });
  const handler = new RegisterUserHandler(
    { user: { create } } as unknown as PrismaService,
    { issue } as unknown as TokenService,
  );

  it('lower-cases the e-mail and returns a token', async () => {
    create.mockResolvedValue({ id: 'u1', email: 'a@b.com' });

    const result = await handler.execute(new RegisterUserCommand('A@B.com', 'Str0ngPassw0rd!'));

    expect(result).toEqual({ accessToken: 'token' });
    expect(create).toHaveBeenCalledWith(
      expect.objectContaining({ data: expect.objectContaining({ email: 'a@b.com' }) }),
    );
  });

  it('maps a unique-constraint error (P2002) to ConflictException', async () => {
    create.mockRejectedValue(
      new Prisma.PrismaClientKnownRequestError('duplicate', { code: 'P2002', clientVersion: '7' }),
    );

    await expect(
      handler.execute(new RegisterUserCommand('a@b.com', 'Str0ngPassw0rd!')),
    ).rejects.toBeInstanceOf(ConflictException);
  });
});
```

apps/api/src/auth/queries/handlers/login.handler.spec.ts (NEW)

```
import { UnauthorizedException } from '@nestjs/common';
import { hash } from 'bcryptjs';
import type { PrismaService } from '../../../prisma/prisma.service.js';
import type { TokenService } from '../../token.service.js';
import { LoginQuery } from '../login.query.js';
import { LoginHandler } from './login.handler.js';

describe('LoginHandler', () => {
  const findUnique = vi.fn();
  const issue = vi.fn().mockResolvedValue({ accessToken: 'token' });
  const handler = new LoginHandler(
    { user: { findUnique } } as unknown as PrismaService,
    { issue } as unknown as TokenService,
  );

  it('returns a token for valid credentials', async () => {
    findUnique.mockResolvedValue({ id: 'u1', email: 'a@b.com', passwordHash: await hash('secret123', 4) });

    await expect(handler.execute(new LoginQuery('A@B.com', 'secret123'))).resolves.toEqual({
      accessToken: 'token',
    });
  });

  it('rejects an unknown e-mail', async () => {
    findUnique.mockResolvedValue(null);

    await expect(handler.execute(new LoginQuery('x@y.com', 'whatever1'))).rejects.toBeInstanceOf(
      UnauthorizedException,
    );
  });

  it('rejects a wrong password', async () => {
    findUnique.mockResolvedValue({ id: 'u1', email: 'a@b.com', passwordHash: await hash('secret123', 4) });

    await expect(handler.execute(new LoginQuery('a@b.com', 'wrong-pass'))).rejects.toBeInstanceOf(
      UnauthorizedException,
    );
  });
});
```

> **Проверка.** `npm run test --workspace=api` показывает новые тесты зелёными. Если ESLint ругается на форматирование, запусти `npm run format` из корня.

Шаг 11. Обновить документацию и закоммитить

Правило из твоего `CLAUDE.md`: при изменении модулей надо обновить `apps/api/CLAUDE.md` **в том же изменении**. Раздел «Architecture and routes» сейчас описывает «Standard Nest module/controller/service layout». Замени строку про `src/auth/` примерно так (по-английски):

apps/api/CLAUDE.md (EDIT)

```
- `src/auth/` - `AuthModule`, implemented with CQRS (`@nestjs/cqrs`):
  - `commands/` - `RegisterUserCommand` + `RegisterUserHandler` (writes a user, returns a JWT)
  - `queries/` - `LoginQuery` + `LoginHandler` (verifies credentials, returns a JWT)
  - `token.service.ts` - signs JWTs; `dto/` - request validation
  - `auth.controller.ts` only dispatches to `CommandBus` / `QueryBus`
```

```
npm run format
npm run lint && npm run build && npm run test && npm run test:e2e --workspace=api
git add -A
git commit -m "refactor(api): migrate auth module to CQRS"
```

> **Заметка.** Делай коммиты мельче, если хочется. Удобные точки: после шага 3 (общие куски), после шага 5 (handlers), после шага 8 (переключение и удаление), после шага 10 (тесты).

Итоговая шпаргалка: что установить, добавить, изменить, удалить

Действие	Что именно
Установить	`npm install @nestjs/cqrs --workspace=api` (больше ничего)
Добавить (9 файлов + 2 теста)	`auth/auth.constants.ts`, `auth/auth.types.ts`, `auth/token.service.ts`, `auth/commands/register-user.command.ts`, `auth/commands/handlers/register-user.handler.ts`, `auth/commands/index.ts`, `auth/queries/login.query.ts`, `auth/queries/handlers/login.handler.ts`, `auth/queries/index.ts`; тесты `register-user.handler.spec.ts` и `login.handler.spec.ts`
Изменить	`app.module.ts` (+`CqrsModule.forRoot()`), `auth/auth.module.ts` (providers), `auth/auth.controller.ts` (шины вместо сервиса), `apps/api/CLAUDE.md`
Удалить	`auth/auth.service.ts`
Не трогать	`dto/auth-credentials.dto.ts`, `main.ts`, `prisma/*`, `test/auth.e2e-spec.ts`, оба vitest-конфига, `tsconfig.json`

Чек-лист

- [ ] Шаг 0: e2e зелёные до начала, ветка создана
- [ ] Шаг 1: `@nestjs/cqrs` установлен
- [ ] Шаги 2–3: папки, константа, тип, `TokenService`
- [ ] Шаг 4: `RegisterUserCommand` + `RegisterUserHandler` + `commands/index.ts`
- [ ] Шаг 5: `LoginQuery` + `LoginHandler` + `queries/index.ts`
- [ ] Шаг 6: build и lint без ошибок
- [ ] Шаг 7: `AppModule`, `AuthModule`, `AuthController` переписаны подряд
- [ ] Шаг 8: `auth.service.ts` удалён
- [ ] Шаг 9: build, lint, `test:e2e` зелёные
- [ ] Шаг 10: юнит-тесты handlers
- [ ] Шаг 11: `apps/api/CLAUDE.md` обновлён, коммит сделан

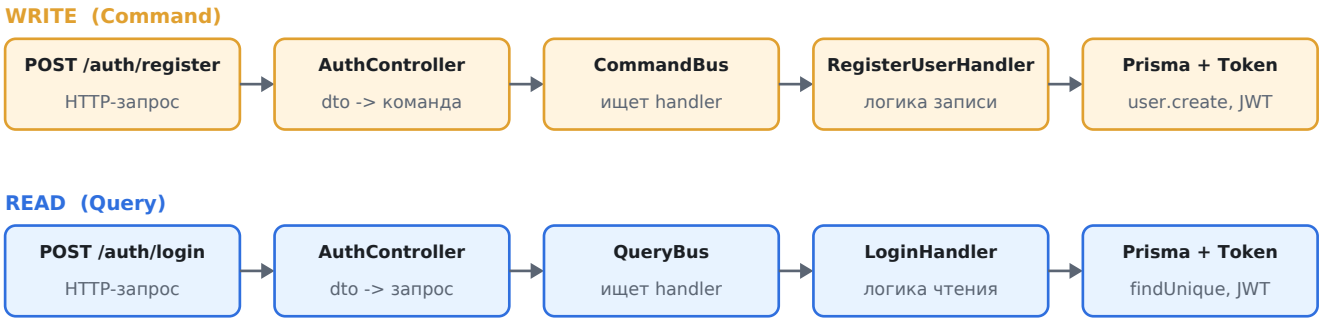
Куда расти дальше (необязательно)

- **Запрос** `GET /auth/me`: `GetCurrentUserQuery` + handler плюс JWT-guard. Первая настоящая «чистая» query, которой стоит иметь отдельный read-handler.
- **События**: после регистрации публиковать `UserRegisteredEvent` через `EventBus` (или `AggregateRoot`) и подписать на него `@EventsHandler`. Туда можно повесить приветственное письмо, не усложняя `RegisterUserHandler`.
- **Новые модули видеовстреч** (`meetings`, `participants`): сразу строй их по той же схеме `commands/ queries/ handlers/`, как этот модуль.
- **Saga** (`@Saga`) пригодится, когда понадобится цепочка «событие -> следующая команда» (например, создание комнаты после подтверждения встречи).

Схема: что добавляем по порядку (по паттерну CQRS)

Порядок построен «снизу вверх»: сначала то, от чего зависят другие файлы, потом сообщения (команда и запрос), потом их обработчики, потом регистрация и в самом конце точка входа. Так проект компилируется после каждого шага.

Как запрос проходит по готовой схеме



Ответ идёт обратно по той же цепочке: handler возвращает { accessToken }, контроллер отдаёт 201.

Порядок создания (шаги 3–8 основного гайда)

№	Слой CQRS	Что создаём	Файл в `apps/api/src/auth/`	Зачем именно сейчас
1	Общие кирпичики	Константа, тип результата, сервис токена	`auth.constants.ts`, `auth.types.ts`, `token.service.ts`	От них зависят оба handler. Пока нигде не подключены, приложение не ломается.
2	**Command** (сообщение, пишет)	`RegisterUserCommand`: email, password	`commands/register-user.command.ts`	Описываем намерение «зарегистрировать». Только данные, без логики.
3	**Command** handler	`RegisterUserHandler`: bcrypt, `user.create`, 409 на `P2002`, токен	`commands/handlers/register-user.handler.ts`	Сюда переезжает `AuthService.register`. Handler знает свою команду.
4	Регистрация команд	Массив `CommandHandlers`	`commands/index.ts`	Одна точка для подключения всех handler команд в модуль.
5	**Query** (сообщение, читает)	`LoginQuery`: email, password	`queries/login.query.ts`	Описываем запрос «проверь логин и выдай токен». Данные без логики.
6	**Query** handler	`LoginHandler`: `findUnique`, `compare`, 401, токен	`queries/handlers/login.handler.ts`	Сюда переезжает `AuthService.login`.
7	Регистрация запросов	Массив `QueryHandlers`	`queries/index.ts`	Аналогично шагу 4, но для read-стороны.
8	Подключение (wiring)	`CqrsModule.forRoot()`, providers модуля	`app.module.ts`, `auth.module.ts`	Без этого шины не найдут handlers (`HandlerNotFoundException`).

№	Слой CQRS	Что создаём	Файл в `apps/api/src/auth/`	Зачем именно сейчас
9	Точка входа	`CommandBus` и `QueryBus` в контроллере	`auth.controller.ts`	Последнее: контроллер начинает слать сообщения в шины.
10	Уборка	Удалить старый сервис	`auth.service.ts` (delete)	Только когда никто на него не ссылается и e2e зелёные.

Запомнить. Золотое правило CQRS в этом проекте: **контроллер ничего не решает**. Он берёт DTO, делает из него сообщение (`Command` или `Query`) и отправляет в шину. Вся логика живёт только в handler, а у каждого сообщения ровно один handler.

Куда класть новую фичу в будущем (шаблон на 5 действий)

1. Решить: фича **меняет** состояние (Command) или только **читает** (Query).
2. Создать класс сообщения: `<Имя>Command` или `<Имя>Query` (только поля, без логики).
3. Создать handler с `@CommandHandler(...)` или `@QueryHandler(...)` и методом `execute`.
4. Добавить handler в массив `CommandHandlers` / `QueryHandlers` (он попадёт в `providers` автоматически).
5. В контроллере вызвать `commandBus.execute(...)` или `queryBus.execute(...)`. Написать unit-тест handler и e2e-тест роута.